

Selective Observation for Browser Agents

Benchmark-first selective observation with a local diagnostic suite and a curated MiniWoB++ external benchmark

Final project report
June 2026

Insert team photo Name	Insert team photo Name	Insert team photo Name
Replace with actual team photo and name.	Replace with actual team photo and name.	Replace with actual team photo and name.

Note: team photos and exact member names were not present in the repository, so this page includes explicit placeholders for final submission.

Introduction

This project studies a specific question in browser-based GUI agents: when is a fresh observation really necessary? Many agents re-read the page before every action, which increases latency and model cost. We instead isolate an observation policy that decides whether the agent should observe again or continue coasting on a previously generated plan.

The project ultimately became stronger after a benchmark-first refactor. The final system supports both a controlled local diagnostic suite and a curated subset of MiniWoB++, a recognized external web-agent benchmark. Our strongest result is on the external benchmark: ``learned@0.8`` reaches 0.900 success rate with 2.0 average model calls and 0 unsafe rate, outperforming both the safe baselines (``always`` and ``handrule``) on this curated subset. The local suite provides the complementary negative result: the learned gate mostly collapses toward frequent observation because the cheap-feature labels are highly skewed toward ``observation needed``.

Model and Data

The core agent is an OpenAI-backed browser planner that receives the current task, a short page-status string, and a compact set of visible interactive elements. It outputs a short action sequence with action type, selector, risk label, confidence, and a self-reported ``needs_observation`` flag. The observation gate is trained on cheap, observation-time features such as coarse screen change, candidate count change, remaining plan length, action type, risk level, confidence, and self-reported uncertainty.

- Local diagnostic suite: 17 tasks spanning static baselines, async results, value drift, and delayed options.
- MiniWoB curated suite: 10 external benchmark instances covering click, focus, radio, checkbox, and sequence tasks.
- Training data: two DAgger-style collection rounds, one under ``always`` and one under ``learned@0.5``.

Technical Approach

We restructured the project around three layers: (1) the observation-policy core, (2) environment adapters, and (3) task-suite manifests. The adapter layer now exposes `reset/observe/execute/success` operations for both the local Playwright environment and the MiniWoB Playwright environment. This was the main architectural change that turned the project from a local demo into a framework that can plausibly migrate to stronger benchmarks such as WebArena or WorkArena.

1. Collect oracle-labeled trajectories under ``always`` observe.
2. Train a lightweight gate (``logistic`` or one-hidden-layer ``MLP``) on the logged cheap features.
3. Run another collection round under the current learned gate to reduce policy-induced distribution shift.

4. Sweep multiple policies at evaluation time: `always`, `never`, `handrule`, and `learned@tau`.

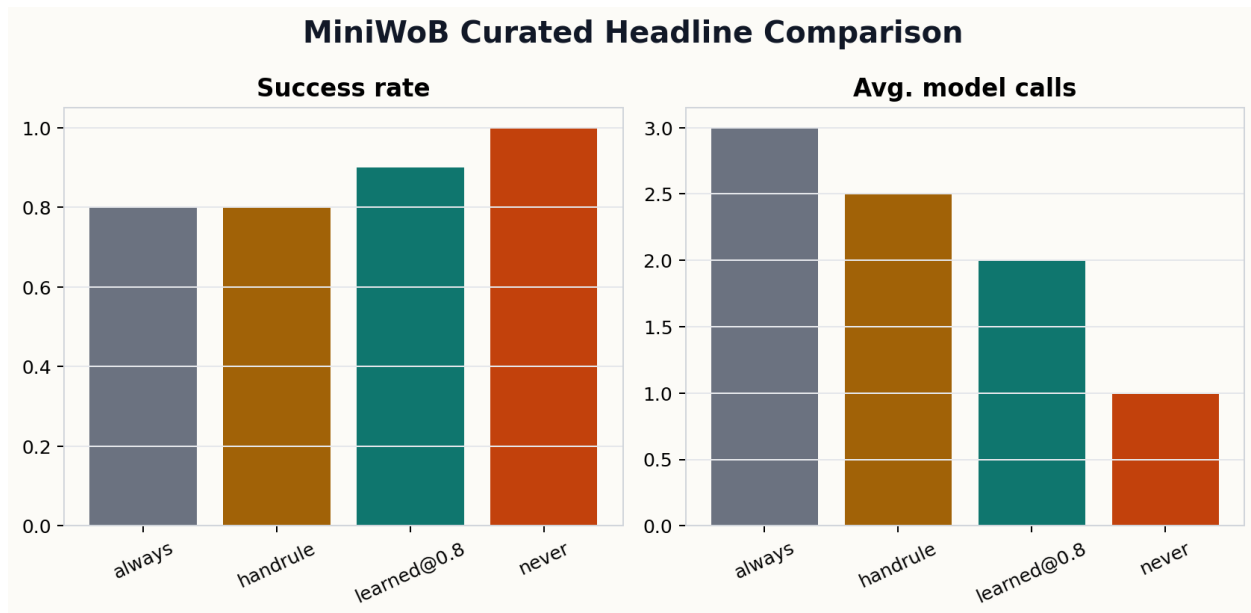
The best selected gate remained logistic. On the aggregated local training data, the label base rate was 0.9339, which already hints at the project's central challenge: the diagnostic suite often makes observation genuinely necessary.

Results and Discussion

We report two complementary result sets. The external benchmark is the headline experiment because it gives the project a recognizable evaluation target. The local diagnostic suite is the failure-analysis environment because it explicitly stresses observation-sensitive failure modes.

External benchmark headline comparison (MiniWoB curated)

Policy	Success	Avg. calls	Unsafe rate	Comment
always	0.800	3.00	0.000	Safe but expensive; poor on the local suite, solid on MiniWoB.
handrule	0.800	2.50	0.000	Best safe hand-coded baseline.
learned@0.8	0.900	2.00	0.000	Headline learned policy used in the discussion.
never	1.000	1.00	0.500	Very cheap, but unsafe when checkpoints are skipped.



The external benchmark result is the strongest evidence for the project’s core idea. ``learned@0.8`` improves success from 0.800 to 0.900 over the safe baselines while also reducing average model calls from 3.0 (``always``) and 2.5 (``handrule``) down to 2.0. Importantly, it does this with 0 unsafe rate. The unsafe but cheap ``never`` policy reaches 1.000 success on this curated subset, but it incurs a 0.500 unsafe rate and is therefore not a realistic deployment choice.

A useful task-level insight appears in the MiniWoB ``click-button-sequence`` instances. After button ONE is clicked, the page does not visibly encode that progress. If the policy re-observes and replans from scratch, it may forget that ONE is already complete. In those cases, selective coasting can be more reliable than fresh observation.

Local diagnostic suite headline comparison

Policy	Success	Avg. calls	Unsafe rate	Comment
always	0.118	11.12	0.000	Safe but expensive; poor on the local suite, solid on MiniWoB.
handrule	0.176	9.29	0.000	Best safe hand-coded baseline.
learned@0.8	0.176	10.18	0.125	Headline learned policy used in the discussion.

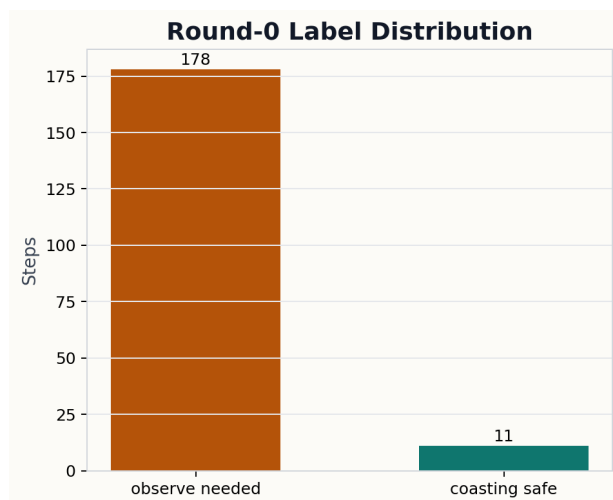
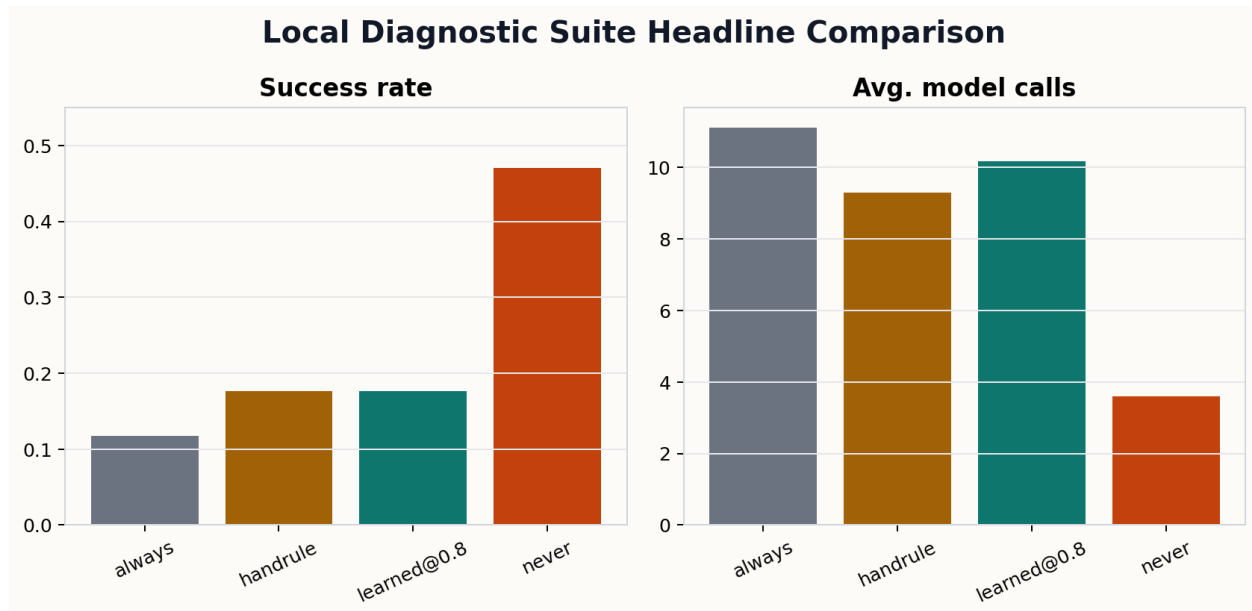
never

0.471

3.59

0.846

Very cheap, but
unsafe when
checkpoints are
skipped.



The local diagnostic suite tells the opposite story, and that negative result is important. The round-0 label distribution is extremely skewed: 178 out of 189 steps require observation. As a result, the learned gate mostly collapses toward frequent observation and does not create a meaningful safe cost reduction. `learned@0.8` only reaches 0.176 success rate, which is similar to `handrule`, while `never` is much cheaper but dangerously unsafe (0.846 unsafe rate).

This is not evidence that selective observation is a bad idea. Instead, it shows that a gate trained only on cheap single-step features is not expressive enough for the hardest controlled stress tests. The local suite therefore serves as a useful research diagnostic: it reveals where richer state features, memory, or better sequence-level abstractions are needed.

Conclusions

The project is strong enough as a final project because it is no longer just a browser-agent demo. It now asks a clear research question, implements a reusable benchmark-first framework, and produces both a positive external-benchmark result and a meaningful negative diagnostic result. The main lesson is that selective observation can work on a recognized benchmark, but its current feature set is too weak for extremely observation-sensitive local stress cases.

- What we learned: benchmark-first architecture matters almost as much as the gate itself.
- Main limitation: the local diagnostic labels are highly imbalanced toward ‘observe’.
- Future work: migrate the same adapter core to WebArena or WorkArena, add richer state/memory features, and collect more balanced trajectories.

Contributions

The exact team-member mapping was not present in the repository, so the table below is a fill-in template for the final submission.

Team member	Primary contribution	Collaboration notes
Member 1	Project framing, benchmark-first refactor, experiment design	Worked on the shared agent and evaluation loop
Member 2	Task design, diagnostics, data collection, analysis	Validated results and helped iterate on failure cases
Member 3	Reporting, slides, documentation, polishing	Integrated visuals and final submission materials

Prompt Appendix

AI tools were used for coding, debugging, experiment orchestration, writing, and presentation production. The workflow remained human-directed: prompts were used to inspect the repository, critique the benchmark story, implement adapters and evaluation changes, and draft submission materials.

- Key coding prompt: “把 benchmark 变成系统的一等公民，而不是未来计划。” This drove the adapter refactor.
- Key evaluation prompt: “帮我把实验跑了，然后把 ppt 和报告按要求写了。” This triggered the full experiment-and-deliverables workflow.
- Key critique prompt: “toy task 怎么改？有别的论文的 task 吗？” This shifted the framing toward recognized benchmarks and diagnostic stress suites.

- Key polishing prompt: “你这样改是不是有点像补丁啊？” This motivated the benchmark-first architecture instead of a patch-like migration plan.

In addition, the project itself uses an LLM planner at runtime. The central agent system prompt asks the model to output short action plans using only visible candidates, to tag risky actions, and to report confidence and `needs\_observation`. That runtime prompt is part of the system under study rather than a documentation-only writing aid.